

IF 260

Sistem Operasi

Minggu 4

Dr. Ananda Kusuma
e-mail: ananda.kusuma@lecturer.umn.ac.id

Universitas Multimedia Nusantara
Serpong, Tangerang

Agenda

- **Review**
- **Topik Minggu 4:**
 - **Interprocess Communications (Komunikasi antar Process)**
 - **Permasalahan sinkronisasi antar process/thread**
 - Race condition
 - Upaya mengatasi masalah:
 - Mutual exclusion
 - Busy waiting
 - **Classic problems**
 - **Producer-Consumer Problem**
 - Sinkronisasi dengan sleep dan wakeup
 - **Dining Philosophers Problem**

Review:
- Process & Thread

Review

- **Pengertian Process?**
- **Jelaskan three-state model (state, transition dan event)**
- **Context switching**
- **CPU-bound vs IO-bound process**
- **Pengertian Thread?**
- **Kenapa Thread diperlukan?**

Contoh program: process creation using fork

```
...
int main()
{
    ...
    childPID = fork();
    if (childPID == 0)
        printf("In child process\t %d\t%d\n",parentPID,getpid());
    else
        printf("In parent process\t %d\t%d\n",parentPID,childPID);

    printf("Exit.....\n");
}
```

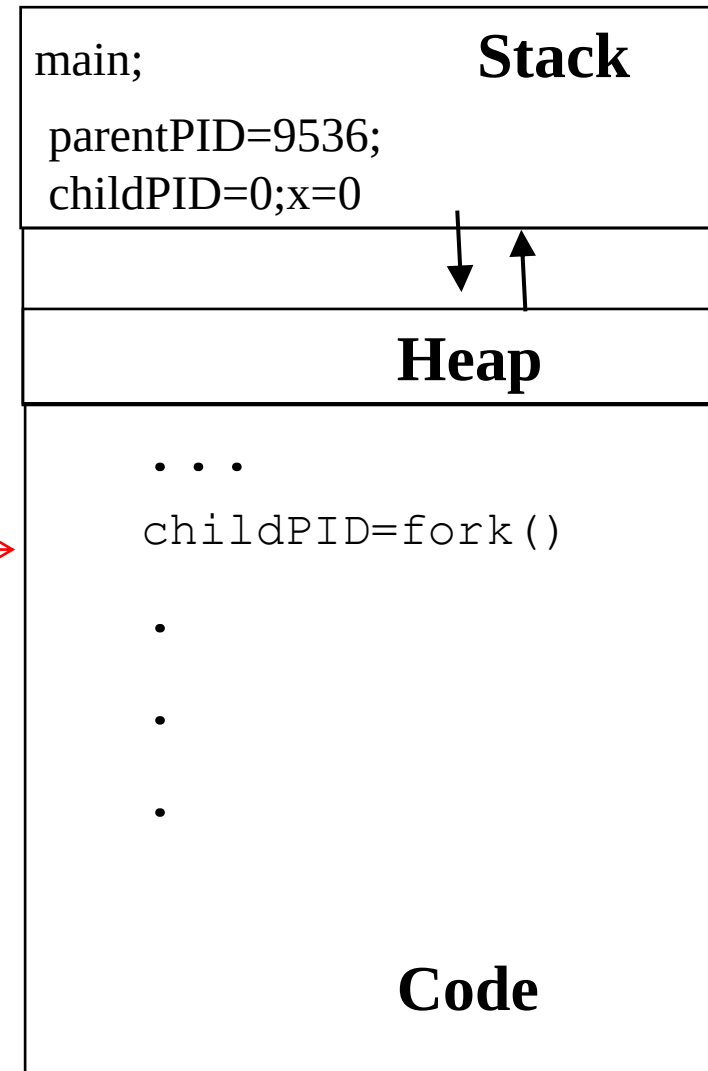
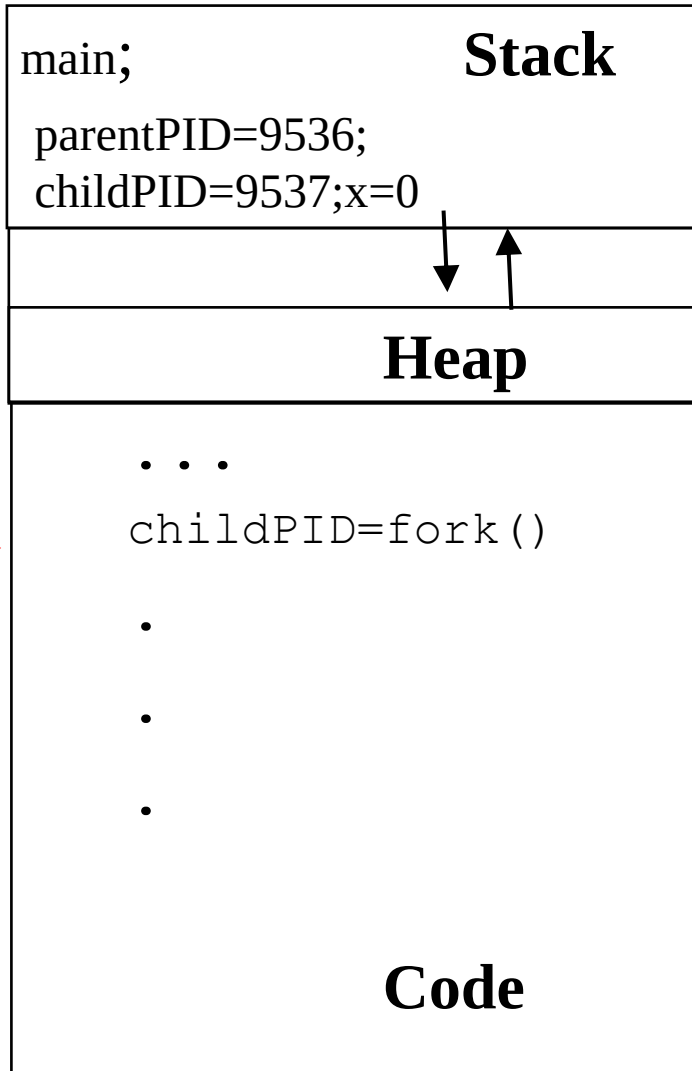
Berapa kali “Exit.....” muncul di layar saat program dijalankan?

Address spaces saat fork() dieksekusi

Copy semuanya, kecuali return value dari fork()

Parent

Child



POSIX Threads API

- IEEE mendefinisikan standard untuk threads pada IEEE standard 1003.1c.
- Salah satu package library yang tersedia adalah pthread, dengan beberapa fungsinya sebagai berikut:

`pthread_create()`

`pthread_exit()`

`pthread_join()`

`pthread_yield()`

`pthread_attr_init()`

`pthread_attr_destroy()`

Contoh program: thread creation

```
...
int g1 = 0;

void task1(int *counter) {
    ...
    g1 = 10; }

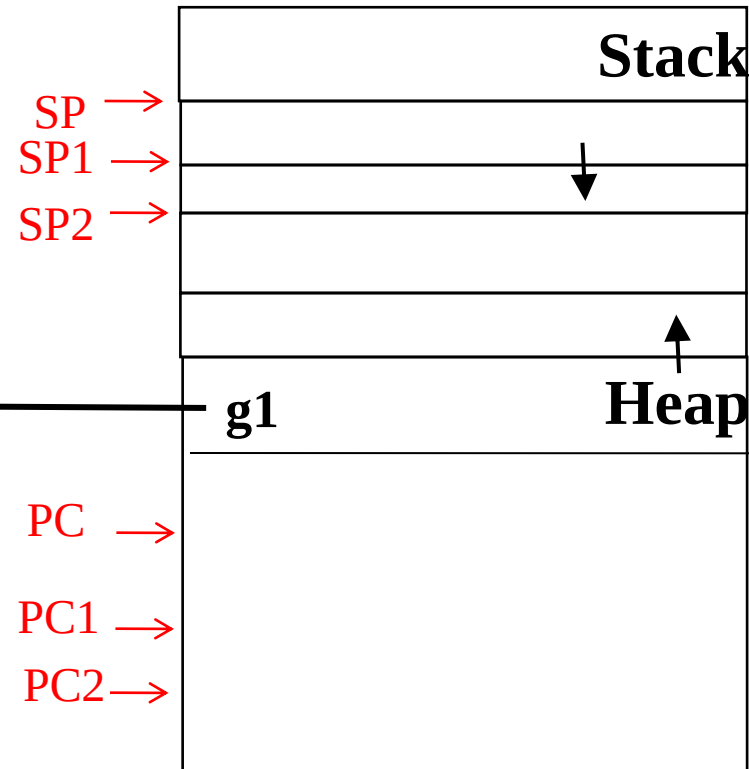
void task2(int *counter) {
    ...
    g1 = 20; }

int main()
{
    pthread_t thrd1,thrd2;
    int a=1,b=2,ret;
    ...
    ret = pthread_create(&thrd1,NULL,(void *)task1,(void *)&a);
    ret = pthread_create(&thrd2,NULL,(void *)task2,(void *)&b);
    ...
    pthread_join(thrd2,NULL);
    pthread_join(thrd1,NULL);
}
```

Address space

Global variable digunakan oleh ketiga threads

Variable g1 dapat digunakan
oleh ketiga threads
Nilai akhir tergantung urutan
eksekusi threads
→ Race condition



Interprocess Communications (IPC)

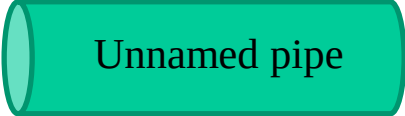
Pendahuluan

- **Processes perlu bekerja sama untuk efisiensi penggunaan sistem**
- **Karena tiap process merupakan entitas independen yang dilindungi oleh kernel, diperlukan mekanisme yang diatur kernel untuk**
 - **Penyaluran informasi antar process**
 - **Pengaturan process agar tidak saling menghalangi**
 - **Pengurutan (proper sequencing) bila ada saling ketergantungan**
- **IPC dapat dilakukan via file, signal, socket, message queue, pipe, named pipe, semaphore, shared memory, message passing**
- **Bagaimana dengan komunikasi antar threads?**

Contoh IPC:

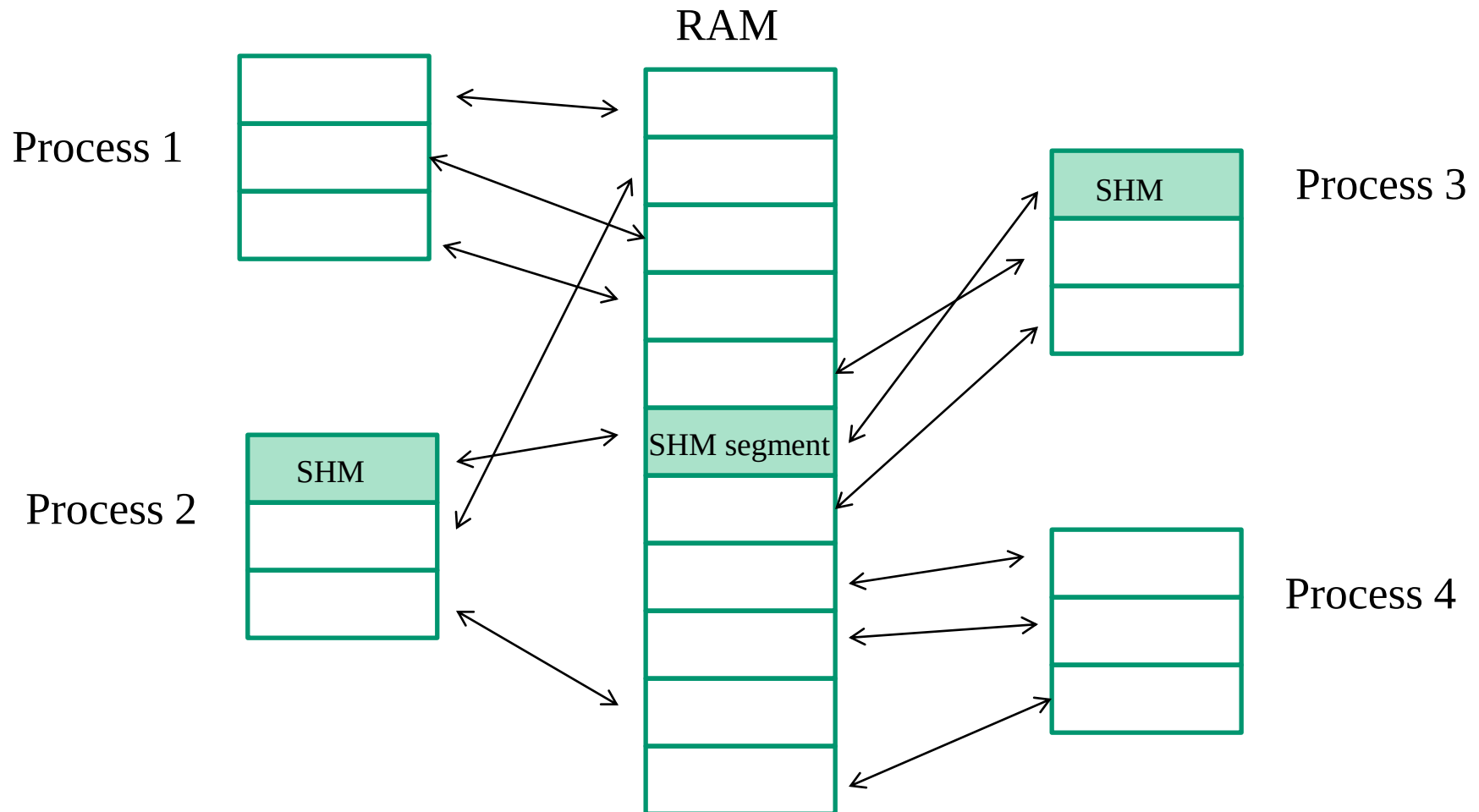
unnamed pipe dan named pipe (FIFO)

- “cat /etc/passwd | cut -f1 -d:”
- “cut -f1 -d: < /tmp/myfifo” dan “cat /etc/passwd > tmp/myfifo”

cat /etc/passwd →  → cut -f1 -d:

cat /etc/passwd > /tmp/myfifo →  → cut -f1 -d: < /tmp/myfifo

Contoh IPC: Shared memory

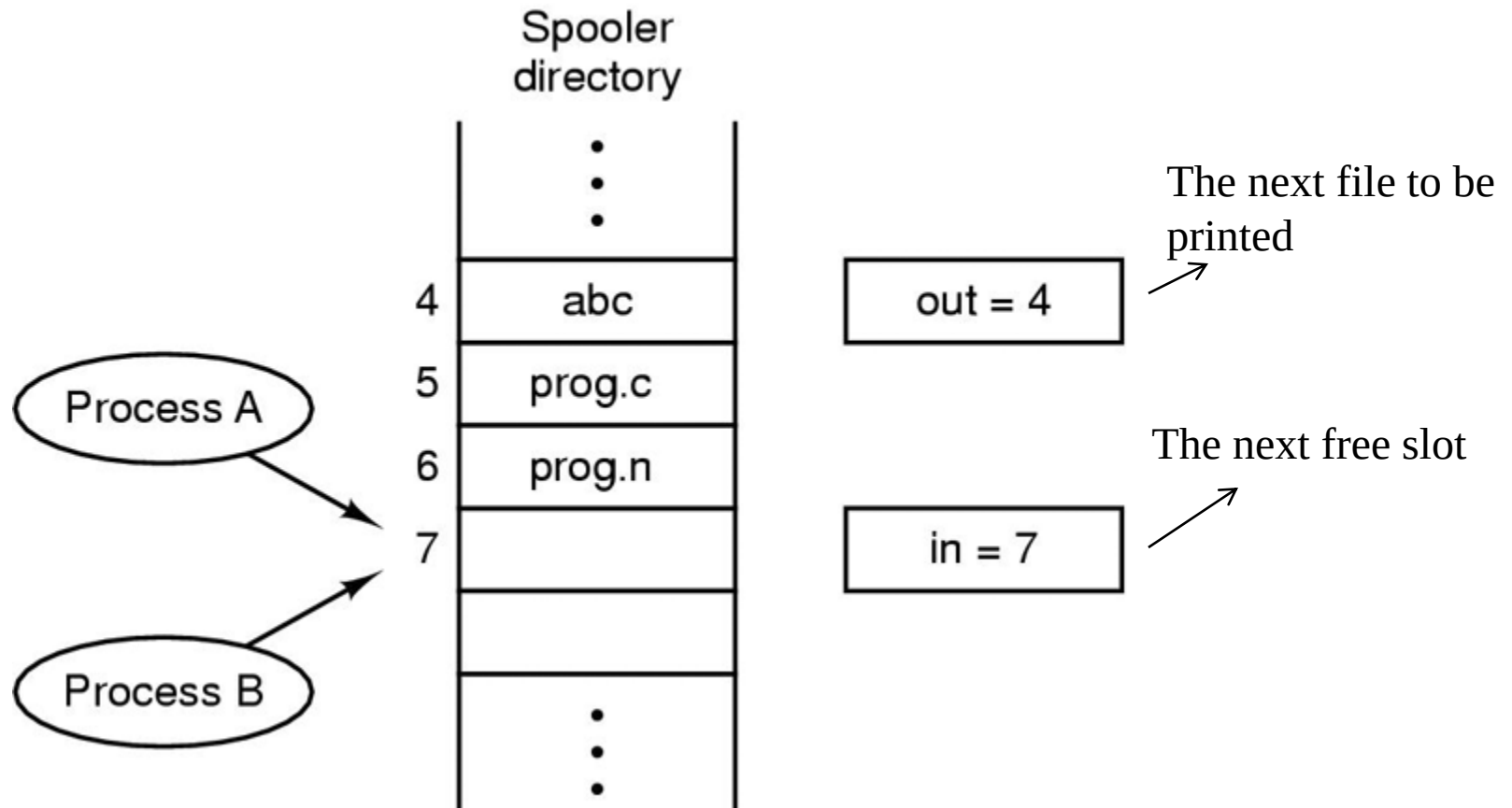


Race Condition

- Terdapatnya processes yang bekerja pada shared media/resources (memory, file system, i/o port, dsb) secara concurrent dapat menimbulkan masalah → **race condition**
- **Race condition** terjadi apabila hasil dari suatu operasi ditentukan oleh urutan (ordering) dari process pada data yang di-shared
 - Penjadwalan (scheduling) process sendiri dikendalikan oleh CPU, sehingga race condition ini memberikan hasil operasi yang sifatnya non-deterministic di sisi pengguna
 - Mempersulit debugging karena errors intermittent (kadang muncul, kadang tidak)
 - Programmer perlu memahami di mana problem ini dapat muncul dan mempersiapkan langkah-langkah antisipasi

Ilustrasi Race Condition: Printer spooler

- Printer daemon secara periodik memeriksa spooler directory untuk mengambil file yang akan di-print
- Shared data?



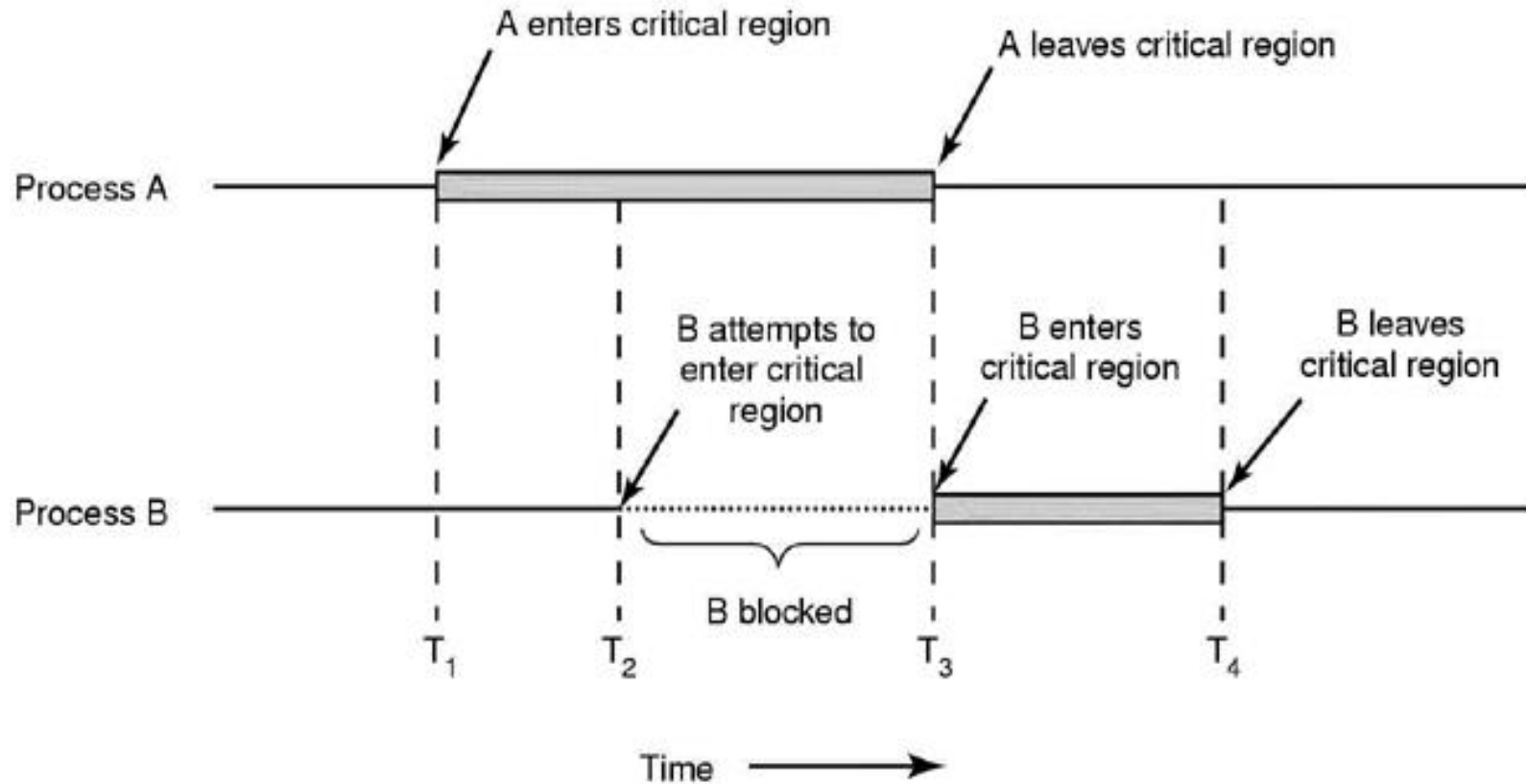
Critical Region/Section

- Bagian dari program di mana terdapat shared data/resources yang apabila diakses secara bersamaan oleh beberapa process dapat menghasilkan hasil yang non-deterministic/inconsistent atau yang tidak diinginkan seperti deadlock, starvation.
- Untuk itu diperlukan suatu kondisi agar tidak ada process yang mempengaruhi suatu data/resource yang sedang dipakai oleh suatu process atau dengan kata lain penggunaan eksklusif
 - **Mutual Exclusion**

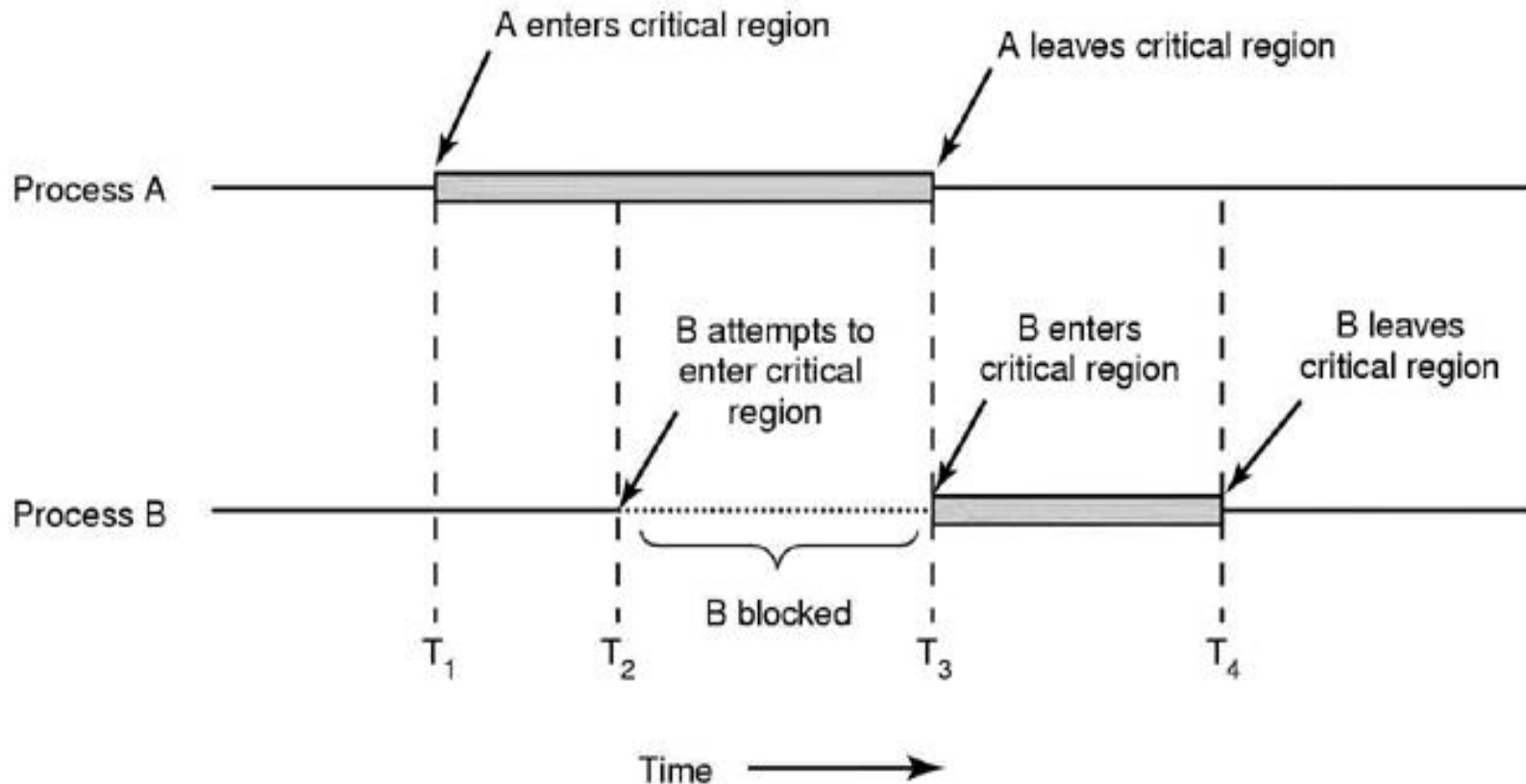
Critical Region: kondisi ideal

- **Tidak ada dua process yang berada di critical region pada saat yang bersamaan**
- **Tidak ada asumsi mengenai kecepatan dan jumlah CPU**
- **Tidak ada process yang berada di luar critical region menghalangi (blocking) process yang lain**
- **Tidak ada process yang menunggu selamanya untuk masuk ke critical region**

Mutual Exclusion: ilustrasi



Mutual Exclusion: ilustrasi



Bagaimana kalau kita gunakan shared variable lock sebelum masuk ke critical region?
Jika lock=0 bisa masuk critical region
Jika lock=1 tidak bisa masuk critical region

Bagaimana kalau process yang masuk critical region langsung disable interrupts dan enable interrupts sebelum keluar critical region?

Mutual exclusion with busy waiting/spinlock

Strict Alternation (Bergantian masuk critical region)

Kalau $turn = 0$, giliran Process 0 masuk critical region
Kalau $turn = 1$, giliran Process 1 masuk critical region
Terus menerus (busy waiting) periksa nilai $turn$ (maka disebut spinlock)

```
while (TRUE) {  
    while (turn != 0)    /* loop */ ;  
    critical_region();  
    turn = 1;  
    noncritical_region();  
}
```

(a)

(a) Process 0.

```
while (TRUE) {  
    while (turn != 1)    /* loop */ ;  
    critical_region();  
    turn = 0;  
    noncritical_region();  
}
```

(b)

.(b) Process 1.

Kelemahan: Apabila process yang satu jauh lebih lambat dibandingkan process yang lain

Mutual exclusion with busy waiting/spinlock

Peterson's Solution:

Menggunakan ide bergantian masuk critical region,
namun tidak strict (kaku)

```
#define FALSE 0
#define TRUE 1
#define N      2                /* number of processes */

int turn;                       /* whose turn is it? */
int interested[N];              /* all values initially 0 (FALSE) */

void enter_region(int process); /* process is 0 or 1 */
{
    int other;                  /* number of the other process */

    other = 1 - process;        /* the opposite of process */
    interested[process] = TRUE; /* show that you are interested */
    turn = process;             /* set flag */
    while (turn == process && interested[other] == TRUE) /* null statement */ ;
}

void leave_region(int process) /* process: who is leaving */
{
    interested[process] = FALSE; /* indicate departure from critical region */
}
```

Busy waiting

- **Kekurangan:**
 - **Menghabiskan CPU time hanya untuk menunggu kesempatan masuk ke critical region**
 - **Priority inversion problem:**
 - High-priority process berjalan tanpa kemajuan selamanya, karena low-priority process yang kebetulan berada di critical region tidak dapat menyelesaikan tugasnya di critical region karena tidak mendapatkan cpu time
- **Idealnya ada mekanisme suatu process untuk sleep/blocked apabila tidak dapat masuk critical region, dan dibangun (wakeup) apabila sudah bisa masuk ke critical region**
 - **Sleep/wakeup signal, semaphore, message passing (kuliah-kuliah selanjutnya)**

Classic Problem: Producer-Consumer Problem

Producer-Consumer (Bounded-Buffer) Problem

- **Salah satu contoh classic sinkronisasi antar dua jenis processes atau threads:**
 - **Producer:** membuat data dan menyimpan di shared buffer
 - **Consumer:** mengambil data dari shared buffer
- **Terdapat shared buffer dengan kapasitas yang terbatas**
 - **Producer tidak dapat menyimpan tambahan data jika buffer sudah penuh → producer *sleep***
 - **Consumer tidak dapat mengambil data jika buffer kosong → consumer *sleep***
 - **Producer di-wakeup oleh consumer jika ada ruang kosong di buffer**
 - **Consumer di-wakeup oleh producer apabila ada data di buffer**
- **Apabila kedua process tidak sinkron (race condition), dapat berpotensi deadlock (misal kedua process menunggu untuk di-wakeup)**

Producer-Consumer Problem

Contoh penggunaan

- **Penggunaan pipe (circular buffer yang memungkinkan 2 processes berkomunikasi)**
- **GUI events dari keyboard, mouse, dll., dimasukkan ke antrean di buffer oleh OS, untuk kemudian di-consumed oleh aplikasi**
- **Dan lain-lain mekanisme buffering untuk komunikasi I/O**

Producer and Consumer Problem dengan sleep and wakeup system call

Race
Condition?

Shared variable
apa yang tidak
dilindungi?

Adakah
kemungkinan
wakeup message
yang hilang?

```
#define N 100
int count = 0;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        if (count == N) sleep();
        insert_item(item);
        count = count + 1;
        if (count == 1) wakeup(consumer);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        if (count == 0) sleep();
        item = remove_item();
        count = count - 1;
        if (count == N - 1) wakeup(producer);
        consume_item(item);
    }
}
```

Process/Thread
producer

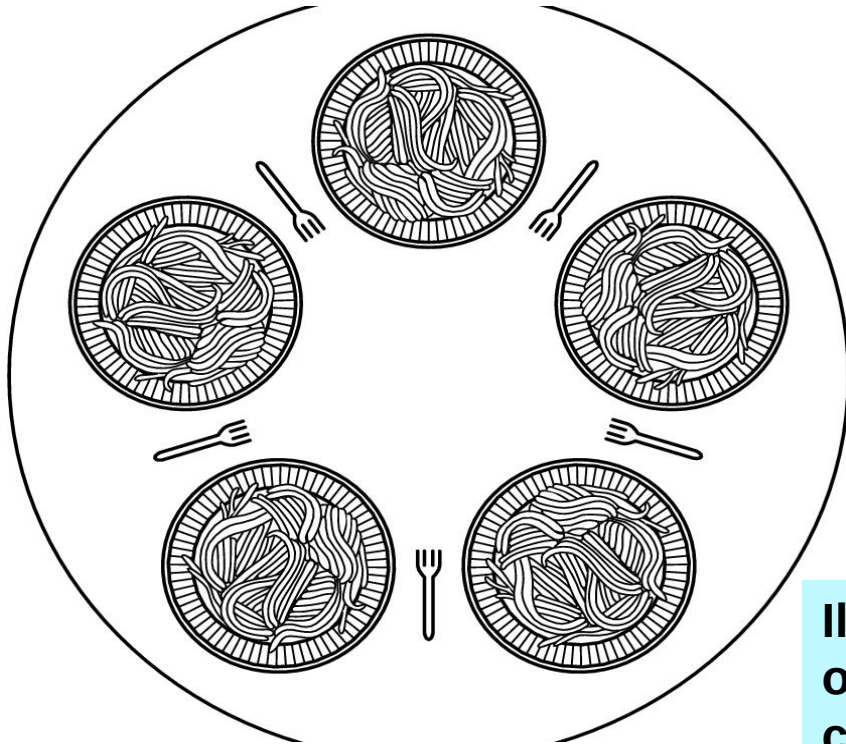
Process/Thread
consumer

Sleep: system call yang menyebabkan proses yang memanggil diblocked (*suspended*)

Wakup: system call yang menyebabkan proses dibangunkan, atau menjadi *ready*

Classic Problem:
Dining Philosophers Problem

Dining Philosophers (1)



Contoh 5 philosophers dan 5 forks

Ilustrasi koordinasi akses shared resources oleh process/thread yang berjalan secara concurrent

**Terdapat N philosophers yang terus menerus Think (berpikir) dan Eat (makan).
Tiap kali lapar, ambil fork (garpu) kiri lalu kanan (atau urutan sebaliknya).
Dengan 2 fork (kiri dan kanan), philosopher dapat makan spaghetti.
Dua philosophers tidak bisa menggunakan fork yang sama pada waktu yang sama (mutual exclusion), dan tidak terjadi kasus deadlock dan starvation.**

Dining Philosophers (2)

```
#define N 5                                /* number of philosophers */

void philosopher(int i)                    /* i: philosopher number, from 0 to 4 */
{
    while (TRUE) {
        think( );                        /* philosopher is thinking */
        take_fork(i);                    /* take left fork */
        take_fork((i+1) % N);            /* take right fork; % is modulo operator */
        eat( );                          /* yum-yum, spaghetti */
        put_fork(i);                     /* put left fork back on the table */
        put_fork((i+1) % N);             /* put right fork back on the table */
    }
}
```

5 process/thread menjalankan procedure philosopher()

Apa masalah dari code di atas?

Bagaimana kalau dimodifikasi sehingga setelah ambil left fork, periksa apakah right fork tersedia. Kalau right fork tidak tersedia, taruh left fork, tunggu beberapa lama, dan ulangi lagi pengambilan fork

Bagian mana yang di-shared dan harus dilindungi?

Bagaimana kalau ke-5 statement setelah think dilindungi dengan mutex?

Akhir Kuliah Minggu 4

Terima kasih atas perhatiannya!